

LEMBAR KERJA PRAKTIKUM 11

STRUKTUR DATA BERHIRARKI – BAGIAN 2

A. Tujuan Praktikum

1. Memahami konsep dasar struktur data hirarki khususnya Binary Search Tree (BST) dan prinsip kerjanya.
2. Menganalisis dan melengkapi potongan kode program BST untuk memahami fungsi-fungsi utama seperti pencarian, penyisipan, dan penghapusan.
3. Mengimplementasikan operasi dasar pada BST sesuai aturan yang benar.
4. Menjelaskan dan membedakan kasus-kasus penghapusan node dalam BST beserta prosedurnya.
5. Menganalisis keterbatasan BST dan memahami konsep dasar pengembangan menjadi AVL Tree melalui proses balancing.

B. Praktikum

Perhatikan baris kode berikut:

```
# include <iostream>
# include <cstdlib>
using namespace std;
struct node {
    int info;
    struct node *left;
    struct node *right;
}*root;
```

1

```
class BST {
public:
    void fun_1(int, node **, node **);
    void fun_2(int);
    void fun_3(node *, node *);
    void fun_4(node *, int);
    void case_a(node *, node *);
    void case_b(node *, node *);
    void case_c(node *, node *);
    BST()
    {
        root = NULL;
    };
};
```

```
int main()
{
    int choice, num;
    BST bst;
    node *temp;
    while (1)
    {
        cout<<"-----"<<endl;
        cout<<"Operasi pada Binary Search Tree"<<endl;
        cout<<"-----"<<endl;
        cout<<"....."<<endl;
        cout<<"....."<<endl;
        cout<<"....."<<endl;
        cout<<"....."<<endl;
        cout<<"Ketik angka yang akan anda pilih: ";
        cin>>choice;
        switch(choice)
        {
            case 1:
                cout<<"....."<<endl;
                bst.fun_4(root,1);
                cout<<endl;
                break;
            case 2:
                if (root == NULL){
                    cout<<"Tree kosong"<<endl;
                    continue;}
                cout<<"Ketik angka ... .. dari BST: ";
```

2

```

        cin>>num;
        bst.fun_2(num);
        break;
    case 3:
        temp = new node;
        cout<<"Ketik ... .. ke BST: ";
        cin>>temp->info;
        bst.fun_3(root, temp);
        break;
    case 4:
        exit(1);
    default:
        cout<<"Tidak ada angka tersebut!"<<endl;
    }
}

```

```

void BST::fun_1(int item, node **par, node **loc)
{

```

```

    node *ptr, *ptrsave;
    if (root == NULL){
        *loc = NULL;
        *par = NULL;
        return;}
    if (item ==.....){
        *loc = root;
        *par = NULL;
        return;}
    if (item < root->info)
        ptr =.....;
    else
        ptr =.....;
    ptrsave = root;
    while (ptr != NULL)
    {
        if (item == ptr->info)
        {
            *loc = ptr;
            *par = ptrsave;
            return;
        }
        ptrsave = ptr;
        if (item < ptr->info)
            ptr =.....;
        else
            ptr =.....;
    }
    *loc = NULL;
    *par = ptrsave;
}

```

3

```

void BST::fun_3(node *tree, node *newnode)
{

```

```

    if (root ==.....)
    {
        root =.....;
        root->info = newnode->info;
        root->left = NULL;
        root->right =.....;
    }
}

```

4

```

        cout<<"Node Root ditambahkan ke dalam Tree"<<endl;
        return;
    }
    if (... .. ==.....)
    {
        cout<<"Node yang akan ditambahkan sudah berada di dalam tree"<<endl;
        return;
    }
    if (tree->info > newnode->info)
    {
        if (tree->left != NULL)
        {
            ... .. (tree->left, newnode);
        }
        else
        {
            tree->left = newnode;
            (tree->left)->left =.....;
            (tree->left)->right =.....;
            return;
        }
    }
    else
    {
        if (... .. != NULL)
        {
            fun_3(... .., newnode);
        }
        else
        {
            tree->right =.....;
            (tree->right)->left = NULL;
            (tree->right)->right = NULL;
            return;
        }
    }
}

```

```

void BST::fun_2(int item)
{
    node *parent, *location;
    if (.....)
    {
        cout<<"Tree dalam keadaan empty"<<endl;
        return;
    }
    fun_1(item, &parent, &location);
    if (location == NULL)
    {
        cout<<"Node yang dicari tidak dapat ditemukan di dalam tree"<<endl;
        return;
    }
    if (location->left == NULL && location->right == NULL)
        ... .. (parent, location);
    if (location->left != NULL && location->right == NULL)
        ... .. (parent, location);
    if (location->left == NULL && location->right != NULL)
        ... .. (parent, location);
}

```

```

        if (location->left != NULL && location->right != NULL)
            case_c(parent, location);
        free(location);
    }

```

```

void BST::case_a(node *par, node *loc )

```

```

{
    if (par == NULL)
    {
        root = NULL;
    }
    else
    {
        if (loc == par->left)
            par->left = NULL;
        else
            par->right = NULL;
    }
}

```

6

```

void BST::case_b(node *par, node *loc)

```

```

{
    node *child;
    if (loc->left != NULL)
        child = loc->left;
    else
        child = loc->right;
    if (par == NULL)
    {
        root = child;
    }
    else
    {
        if (loc == par->left)
            par->left = child;
        else
            par->right = child;
    }
}

```

7

```

void BST::case_c(node *par, node *loc)

```

```

{
    node *ptr, *ptrsave, *suc, *parsuc;
    ptrsave = loc;
    ptr = loc->right;
    while (ptr->left != NULL)
    {
        ptrsave = ptr;
        ptr = ptr->left;
    }
    suc = ptr;
    parsuc = ptrsave;
    if (suc->left == NULL && suc->right == NULL)
        case_a(parsuc, suc);
    else
        case_b(parsuc, suc);
    if (par == NULL)
    {
        root = suc;
    }
}

```

8

```

else
{
    if (loc == par->left)
        par->left = suc;
    else
        par->right = suc;
}
suc->left = loc->left;
suc->right = loc->right;
}

void BST::fun_4(node *ptr, int level)
{
    int i;
    if (ptr != NULL)
    {
        fun_4(ptr->right, level+1);
        cout<<endl;
        if (ptr == root)
            cout<<"Root->:  ";
        else
        {
            for (i = 0; i < level; i++)
                cout<<"      ";
        }
        cout<<ptr->info;
        fun_4(ptr->left, level+1);
    }
}

```

9

Setelah Anda mempelajari program di atas, diskusikan pertanyaan terkait program dengan kelompok Anda, lalu buat video diskusinya selama jam praktikum (jika waktunya tidak cukup bisa dilanjutkan setelahnya) dan tuliskan hasil diskusinya (PDF), masukkan ke gdrive, lalu kumpulkan link gdrive pada form berikut: <https://forms.gle/JCwa8XJy9n7Vnc7r7>. Video dan Hasil Diskusi (dikumpulkan selambatnya satu minggu setelah praktikum ini)

1. Pada potongan kode (snippet) nomor 1, jelaskan hasil eksekusi dari baris kode tersebut?
2. Lengkapi potongan kode (snippet) nomor 3, lakukan penelusuran dan jelaskan apa kegunaan dari fungsi tersebut?
3. Lengkapi potongan kode (snippet) nomor 4, lakukan penelusuran dan jelaskan apa kegunaan dari fungsi tersebut?
4. Lengkapi potongan kode (snippet) nomor 5, lakukan penelusuran dan jelaskan apa kegunaan dari fungsi tersebut?
5. Jelaskan procedural dari potongan kode 6, 7, 8. Apa kegunaanya?
6. Lengkapi potongan kode (snippet) nomor 2, dan implementasikan keseluruhan kode agar dapat menjalankan perintah *insertion*, dan *deletion* BST dengan kriteria sebagai berikut:
Insertion: 32 – 11 – 13 – 46 – 78 – 4 – 16 – 8 – 25 – 37 – 56 – 61 – 44 – 29
Deletion: 8 – 46 – 32 – 5
Catatan: *PrtSc* hasil eksekusi kode tersebut.

7. DISKUSI KELOMPOK

Berikan suatu penjelasan dan gambaran/ide yang dapat dilakukan agar kode pada poin A di atas, dapat berfungsi sebagai AVL tree. Prosedur apa yang perlu dilengkapi, bagaimana prosesnya? (jelaskan dengan minimal 100 kata)